



FOR JAVA DEVELOPERS MOVING TO NESTJS

# 19,000 redeliveries. Or one.

Spring Boot to NestJS, episode 28: RabbitMQ with NestJS microservices

Acknowledgement is the real contract. Once the broker thinks a message is finished, failure after that is yours to handle.

A payment that always fails: Spring Boot's listener container redelivered it over 19,000 times in ten seconds. Nest's defaults delivered it once, and the queue was empty afterwards.

With manual acknowledgement the message stayed, and came back after a restart. With a dead letter queue, both stacks parked it after one attempt.

Nest wraps messages as { pattern, data }. Spring sends bare JSON with a type header, and Nest dropped it.

## In scope

- @RabbitListener and @EventPattern
- Acknowledgement and requeue defaults
- Dead letter queues
- The message envelope

## Not in scope

- Exchanges and routing in depth
- Publisher confirms

At the time of recording Nest's RabbitMQ transport defaults to noAck: true.

## 01 The measurements

|                                  |                                            |
|----------------------------------|--------------------------------------------|
| SPRING BOOT DEFAULTS, 10 s       | over 19,000 deliveries; still in the queue |
| NEST DEFAULTS (noAck true)       | 1 delivery; queue empty                    |
| NEST noAck false, ACK ON SUCCESS | kept; redelivered after a restart          |
| DEAD LETTER QUEUE, BOTH          | 1 attempt; parked                          |
| NEST READING SPRING'S MESSAGE    | no pattern: handled 0, gone                |

A payment that always fails, then three good ones.



## 02 Acknowledge on purpose

Turn acknowledgements on in Nest and acknowledge after the handler succeeds; reject without requeue when you give up, and let the queue's dead letter exchange take it.

```
payments.consumer.ts
```

```
@EventPattern('payment.created')
handle(@Payload() payment: Payment, @Ctx() context: RmqContext) {
  const channel = context.getChannelRef();
  const message = context.getMessage();
  try {
    settle(payment);
    channel.ack(message);
  } catch {
    channel.nack(message, false, false); // to the dead letter queue
  }
}
```

with a dead letter queue: one attempt, the failing payment parked, on both stacks

Run on a real machine, 2026-10-01, by scripts/verify-rabbitmq.sh in the course repository.

## 03 Cap the Spring requeue

Spring Boot's listener container requeues a failed delivery by default. Turn that off, or add a retry interceptor, and give the queue a dead letter exchange.

```
application.properties
```

```
spring.rabbitmq.listener.simple.default-requeue-rejected=false
```

Boot defaults: over 19,000 deliveries in ten seconds

Run on a real machine, 2026-10-01, by scripts/verify-rabbitmq.sh in the course repository.

## Before you trust a RabbitMQ consumer

1. Is the message acknowledged before or after your code runs?
2. What happens when the handler throws?
3. Is there a limit on queues?
4. Does every queue have a dead letter queue?
5. Do producer and consumer agree on the envelope?
6. What is the prefetch?

### Everything else

|                                                                                                                                                                                                                            |                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Site</b><br><a href="https://code-with-sam-dev.github.io">https://code-with-sam-dev.github.io</a>                                     |  <b>Facebook</b><br><a href="https://www.facebook.com/codewithsamdev">https://www.facebook.com/codewithsamdev</a>    |
|  <b>YouTube</b><br><a href="https://www.youtube.com/@CodewithSam-Dev">https://www.youtube.com/@CodewithSam-Dev</a>                       |  <b>GitHub</b><br><a href="https://github.com/code-with-sam-dev">https://github.com/code-with-sam-dev</a>           |
|  <b>LinkedIn</b><br><a href="https://www.linkedin.com/company/code-with-sam-dev">https://www.linkedin.com/company/code-with-sam-dev</a> |  <b>Buy me a coffee</b><br><a href="https://buymeacoffee.com/codewithsam">https://buymeacoffee.com/codewithsam</a> |
|  <b>x</b><br><a href="https://x.com/CodeWithSamDev">https://x.com/CodeWithSamDev</a>                                                    |  <b>Buy me sadza</b><br><a href="https://buymesadza.com/codewithsam">https://buymesadza.com/codewithsam</a>        |
|  <b>TikTok</b><br><a href="https://www.tiktok.com/@codewithsamdev">https://www.tiktok.com/@codewithsamdev</a>                           | <b>Every measurement, one command: scripts/verify-rabbitmq.sh</b><br><a href="https://github.com/code-with-sam-dev/spring-to-node">https://github.com/code-with-sam-dev/spring-to-node</a>            |
|  <b>Instagram</b><br><a href="https://www.instagram.com/codewithsamdev">https://www.instagram.com/codewithsamdev</a>                    | <b>NestJS RabbitMQ</b><br><a href="https://docs.nestjs.com/microservices/rabbitmq">https://docs.nestjs.com/microservices/rabbitmq</a>                                                                 |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. RabbitMQ is a trademark of Broadcom. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.